Woxsen University

School of Technology

A

PROJECT REPORT

on

SMARTROUTE: ALGORITHMIC LAST-MILE DELIVERY OPTIMIZATION USING DYNAMIC PROGRAMMING, GREEDY HEURISTICS, DIJKSTRA'S ALGORITHM, AND BELLMAN-FORD

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Computer Science and Engineering

Submitted by:

Himanshu Pandit (WOU/BTECH/CSE/2023/042)
Devyani Pawar (WOU/BTECH/CSE/2023/018)
Heman (WOU/BTECH/CSE/2023/029)
Rimika Jain (WOU/BTECH/CSE/2023/065)

Under the guidance of:

Dr. Syed Javed Pasha
Associate Professor, Department of Computer Science & Engineering
School of Technology, Woxsen University

CERTIFICATE

This is to certify that the project report entitled **"SMARTROUTE: ALGORITHMIC LAST-MILE DELIVERY OPTIMIZATION USING DYNAMIC PROGRAMMING, GREEDY HEURISTICS, DIJKSTRA'S ALGORITHM, AND BELLMAN-FORD"** submitted by **Himanshu Pandit (WOU/BTECH/CSE/2023/042)**, **Devyani Pawar (WOU/BTECH/CSE/2023/018)**, **Heman (WOU/BTECH/CSE/2023/029)**, and **Rimika Jain (WOU/BTECH/CSE/2023/065)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Woxsen University, School of Technology** is a bonafide record of work carried out by the students under my supervision and guidance.

The work embodied in this project report has been carried out by the candidates and has not been submitted elsewhere for a degree.

Signature of Mentor: _____

Name: Dr. Syed Javed Pasha

Designation: Associate Professor, Department of Computer Science & Engineering

Institution: School of Technology, Woxsen University

Date: 13th October 2026

DECLARATION of the candidates

We hereby declare that the project work entitled **"SMARTROUTE: ALGORITHMIC LAST-MILE DELIVERY OPTIMIZATION USING DYNAMIC PROGRAMMING, GREEDY HEURISTICS, DIJKSTRA'S ALGORITHM, AND BELLMAN-FORD"** submitted to the Department of Computer Science and Engineering, School of Technology, Woxsen University, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is our original work and has been carried out under the guidance of **Dr. Syed Javed Pasha**.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature: _____

Name: Himanshu Pandit

Roll Number: WOU/BTECH/CSE/2023/042

Signature: _____

Name: Devyani Pawar

Roll Number: WOU/BTECH/CSE/2023/018

Signature: _____

Name: Heman

Roll Number: WOU/BTECH/CSE/2023/029

Signature: _____

Name: Rimika Jain

Roll Number: WOU/BTECH/CSE/2023/065

Date: 13th October 2026

Place: Hyderabad, Telangana, India

ACKNOWLEDGMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project on *SmartRoute: Algorithmic Last-Mile Delivery Optimization Using Dynamic Programming, Greedy Heuristics, Dijkstra's Algorithm, and Bellman-Ford*.

First and foremost, we extend our heartfelt thanks to our project mentor, **Dr. Syed Javed Pasha**, Associate Professor, Department of Computer Science & Engineering, for his invaluable guidance, continuous support, and constructive feedback throughout the duration of this project. His deep expertise in algorithm design, network optimization, and computational complexity has been instrumental in shaping this research.

We are grateful to the Head of the Department and the administration of the **School of Technology, Woxsen University**, for providing the necessary computational facilities, infrastructure, and academic resources required for this project.

We also express our appreciation to the researchers, open-source contributors, and developers of the Google Maps JavaScript API, Vite, React, and Python communities whose foundational tools supported our empirical and visual implementations.

Finally, we are deeply grateful to our families and peers for their unwavering support, encouragement, and understanding throughout our academic journey.

Himanshu Pandit

Devyani Pawar

Heman

Rimika Jain

ABSTRACT

Urban last-mile logistics represents the most financially intensive and operationally complex component of modern supply-chain networks, accounting for upwards of 53% of total delivery expenditures. Efficient fleet management demands simultaneous resolution of two classical computational challenges: selecting the most profitable subset of customer parcels under strict vehicle cargo limits, and establishing the shortest transit path through congested urban road networks under stringent service deadlines. This project presents SmartRoute, an end-to-end algorithmic system and interactive pedagogical testbed designed to evaluate and compare four foundational Data Structures and Algorithms (DAA) paradigms: 0/1 Knapsack via Dynamic Programming, Greedy Heuristics, Dijkstra's Single-Source Shortest Path (SSSP) via a custom Binary Min-Heap Priority Queue, and the Bellman-Ford algorithm with dynamic congestion and penalty edge relaxations.

The experimental framework models real-world delivery logistics across the metropolitan region of Hyderabad, India, comprising a central distribution depot at the Main Parcel Warehouse (Begumpet) and 35+ customer demand nodes spanning major commercial and residential corridors (including HITEC City, Gachibowli, Banjara Hills, Secunderabad, and Woxsen University). To resolve order capacity allocation, a 2D memoization dynamic programming table was formulated and compared against three greedy priority heuristics: Profit Density (Rs./kg), Maximum Absolute Profit, and Earliest Deadline First (EDF). For spatial path generation, Dijkstra's algorithm was implemented with a manual Binary Min-Heap to guarantee $O((V + E) \log V)$ execution across non-negative road segments, while Bellman-Ford $O(V \cdot E)$ was integrated as a dynamic verification engine to detect negative-weight congestion arbitrage cycles and model variable toll subsidies.

Empirical benchmarks demonstrate that the 0/1 Knapsack Dynamic Programming engine captured Rs. 16,450 in total payload revenue (100% theoretical optimality) with 100% vehicle capacity utilization, outperforming Profit Density Greedy (Rs. 15,200, 92.4% optimal) and Highest Profit Greedy (Rs. 13,800, 83.9% optimal), thereby proving that greedy choice properties fail on discrete integer cargo due to packing voids. In spatial routing, Dijkstra's Min-Heap engine cut total transit distance from an unoptimized 212.4 km down to 142.6 km (a 32.8% distance saving and 24.5% fuel reduction), executing in just 4.2 ms. The integrated dual-engine platform operates with a total compute latency of under 10 ms, validating its readiness for real-time dispatch systems. Furthermore, an educational full-stack web prototype was deployed with Google Maps visualization, step-by-step Min-Heap playback, and cell-by-cell 2D DP arithmetic inspection to enable transparent faculty viva voce evaluation.

Keywords: *Data Structures and Algorithms, 0/1 Knapsack Problem, Dynamic Programming, Greedy Algorithms, Dijkstra's Algorithm, Bellman-Ford Algorithm, Last-Mile Delivery Optimization, Binary Min-Heap, Shortest Path Networks, Educational Visualization.*

TABLE OF CONTENTS

Certificate ii

Declaration of the Candidates iii

Acknowledgment iv

Abstract v

List of Tables vii

List of Figures viii

1. INTRODUCTION 1

1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 2
1.4 Objectives 3
1.5 Dataset Overview 4
1.6 Scope and Limitations 5
1.7 Organization of the Report 5

2. LITERATURE REVIEW / TECHNOLOGY REVIEW 6

2.1 Classical Optimization and Dynamic Programming 6
2.2 Greedy Approximations and Fractional Relaxations 7
2.3 Single-Source Shortest Paths in Road Networks 8
2.4 Dynamic Edge Relaxations and Negative Cycle Detection 9
2.5 Vehicle Routing and Last-Mile Urban Logistics 9
2.6 Multi-Objective Dispatching and Priority Queuing 10
2.7 Interactive Pedagogical Systems for Algorithm Learning 11
2.8 Research Gap and Distinctive Contributions 11

3. METHODOLOGY 13

3.1 System Overview and Dual-Engine Architecture 13
3.2 Graph Modeling of Hyderabad Delivery Network 14
3.3 Cargo Allocation: 0/1 Knapsack DP vs Greedy Comparators 15
3.4 Spatial Routing: Dijkstra Min-Heap & Bellman-Ford Audit 17
3.5 Implementation Details 19

3.6 Evaluation Framework 20

4. RESULTS AND DISCUSSION 21

4.1 Model Performance Comparison 21
4.2 Cargo Packing Analysis and Revenue Capture 22
4.3 Routing Distance and Latency Benchmarks 23
4.4 Working Prototype Demonstration 24
4.5 Heuristic Trap and Error Analysis 26
4.6 Comparison with Baselines and Discussion 27

5. CONCLUSION AND FUTURE WORK 29

5.1 Summary of Contributions 29
5.2 Limitations 30
5.3 Future Work 30
5.4 Concluding Remarks 31

REFERENCES 32

APPENDIX – I: Screenshots of Work Done and Deployment Tools 34

APPENDIX – II: Formatting Guidelines and Technical Specifications 36

LIST OF TABLES

Table 1.1: Hyderabad Urban Network & Delivery Request Dataset Statistics 4

Table 3.1: Algorithmic Complexity, Paradigm, and Mathematical Invariants 16

Table 3.2: Hyperparameter and System Operational Parameters 20

Table 4.1: Multi-Algorithm Empirical Performance and Revenue Comparison 21

Table 4.2: Runtime Execution Latency and Memory Space Benchmarks 24

Table A.1: General Formatting Guidelines and Report Compliance Specifications 36

LIST OF FIGURES

Figure 1.1: SmartRoute Interactive Google Maps Fleet Dashboard Overview 4

Figure 2.1: Theoretical Literature Timeline and Foundational Milestones 10

Figure 3.1: End-to-End Dual-Engine Algorithmic Pipeline Architecture 14

Figure 3.2: 0/1 Knapsack 2D DP Memoization Table and Backtracking Schema 16

Figure 3.3: Manual Binary Min-Heap Priority Queue Operations and Invariant 18

Figure 3.4: Bellman-Ford Iterative Edge Relaxation on Hyderabad Road Graph 19

Figure 4.1: Comparative Revenue Capture Across Cargo Selection Algorithms 23

Figure 4.2: Total Fleet Transit Distance Reduction (Baseline vs Optimized) 24

Figure 4.3: Dijkstra Min-Heap Step Tracer & Step Log in Working Prototype 25

Figure 4.4: 0/1 Knapsack 2D DP Interactive Cell Arithmetic Inspector 26

Figure 4.5: Multi-Algorithm Benchmark Matrix and Faculty Viva Voce Guide 27

Figure A.1: GitHub Repository, Code Architecture, and Deployment Workspace 35

1. INTRODUCTION

The rapid expansion of global e-commerce, on-demand logistics, and urban quick-commerce platforms has placed unprecedented strains on metropolitan transportation networks. Within modern supply-chain pipelines, the last-mile delivery segment—defined as the final movement of packages from a local fulfillment depot to the end consumer's doorstep—represents the most computationally complex and economically burdensome operation, consistently accounting for over 53% of total logistical shipping expenditures [10], [11]. Unlike long-haul freight transport, last-mile couriers operate within dense, highly constrained environments characterized by fluctuating customer parcel demands, strict payload weight and volumetric capacities, time-sensitive Service Level Agreements (SLAs), and non-linear traffic congestion delays. Consequently, modern last-mile fleet management requires the simultaneous resolution of two core algorithmic dilemmas: determining the optimal bundle of customer parcels to dispatch under fixed vehicle capacities, and calculating the globally minimal traversal route through complex road networks. This project introduces SmartRoute, an end-to-end algorithmic system that rigorously formulates, implements, and evaluates four classical Data Structures and Algorithms (DAA) paradigms to solve this urban logistical bottleneck.

1.1 Background

From a theoretical computer science perspective, last-mile delivery management naturally decomposes into two interconnected problems: the discrete resource allocation problem and the single-source shortest path (SSSP) network traversal problem [12], [13]. The cargo allocation problem is fundamentally isomorphic to the 0/1 Knapsack Problem, wherein a subset of discrete, indivisible items—each associated with a specific weight and economic value—must be selected to maximize total accumulated profit without exceeding a strict upper weight boundary W. Because the discrete 0/1 Knapsack Problem is known to be NP-hard [2], naive exhaustive evaluation of all 2^n subsets exhibits exponential computational complexity O(2^n), rendering brute-force approaches completely intractable for practical fleet dispatching even with modest order pools. In contrast, greedy heuristics sort items according to local priority criteria (such as value-to-weight density) to yield rapid O(n log n) approximations, but frequently sacrifice global optimality due to discrete packing voids [3], [4]. Optimal solutions necessitate Dynamic Programming (DP), which leverages Bellman's Principle of Optimality [1] to construct a 2D memoization table in pseudo-polynomial time $O(n \cdot W)$.

Simultaneously, spatial route optimization requires navigating a directed, weighted urban graph G = (V, E), where vertices represent delivery destinations and edge weights quantify

travel distance and transit latency. Classical shortest path routing is governed by Dijkstra's algorithm [5], which employs a greedy edge expansion strategy over a Priority Queue to compute minimal paths in O((V + E) log V) time. However, Dijkstra's algorithm strictly presumes non-negative edge weights (w(u, v) >= 0), failing catastrophically in environments with dynamic toll discounts, green corridor rebates, or congestion arbitrage penalties. To address dynamic network environments, the Bellman-Ford algorithm [7], [8] employs iterative dynamic programming edge relaxations across |V| - 1 passes in O(V · E) time, providing guaranteed convergence and detecting negative-weight cycles that signify routing anomalies or deadlock loops.

1.2 Motivation

The motivation for this project stems from both practical industrial challenges and pedagogical imperatives in computer science education:

- **Industrial Relevance:** In fast-paced urban delivery hubs, dispatch managers must make rapid, cost-effective decisions under extreme volume. Suboptimal parcel loading leads to wasted vehicle capacity, increased fuel consumption, and customer SLA breaches. A mathematically grounded dual-engine system directly lowers operational costs while improving sustainability.
- **Pedagogical Rigor:** In undergraduate Data Structures and Algorithms coursework, students frequently encounter Dynamic Programming, Greedy heuristics, Dijkstra's algorithm, and Bellman-Ford as isolated theoretical constructs. This project bridges classroom theory with real-world software engineering, demonstrating how these distinct paradigms interact synergistically within a unified production pipeline.
- **Evaluation Transparency:** In academic viva voce examinations, algorithm implementations are often obscured behind black-box libraries. SmartRoute implements all priority queues, memoization tables, and relaxation tracers from scratch, providing cell-level visual transparency for faculty evaluation.

1.3 Problem Statement

The primary problem addressed in this project is the design, implementation, and empirical evaluation of an autonomous last-mile delivery management system capable of resolving the Dual Bottlenecks of urban parcel dispatching:

1. **The Combinatorial Cargo Selection Bottleneck:** Given an incoming pool of N discrete customer delivery requests, where each order i is defined by a weight w_i, an economic profit p_i, a geographical coordinate c_i, and an arrival deadline d_i, select an optimal subset of orders S ⊆ {1, ..., N} that maximizes total accumulated profit ∑_{i ∈ S} p_i subject to the strict vehicle payload boundary ∑_{i ∈ S} w_i <= W,

while systematically benchmarking the optimality gap between Dynamic Programming and Greedy approximation heuristics.

2. **The Spatial Route Optimization Bottleneck:** Given the selected parcel subset S and a directed urban road network G = (V, E) centered at a primary distribution depot, determine the minimal-distance, minimal-latency transit path that visits all customer nodes in S, while establishing the runtime trade-offs between Dijkstra's greedy Min-Heap implementation and Bellman-Ford's iterative edge relaxation with negative cycle audit.

1.4 Objectives

The specific research and engineering objectives of this project are:

1. To formulate and implement from scratch a 2D Dynamic Programming 0/1 Knapsack engine that guarantees 100% global optimal revenue capture under vehicle capacity boundaries W.
2. To implement and benchmark three distinct greedy heuristics (Profit Density, Highest Profit, Earliest Deadline First) against Dynamic Programming, quantifying the mathematical impact of greedy traps and packing voids.
3. To engineer a manual Binary Min-Heap Priority Queue to power Dijkstra's algorithm, verifying $O((V + E) \log V)$ shortest path execution across Hyderabad's road network.
4. To implement the Bellman-Ford algorithm in $O(V \cdot E)$ time to model dynamic traffic congestion surcharges and verify the absence of negative-weight routing cycles.
5. To construct an interactive, production-grade web prototype integrating Google Maps, step-by-step Min-Heap playback, and 2D DP memoization arithmetic inspection for faculty viva voce defense.

1.5 Dataset Overview

The empirical evaluation is grounded in an authentic geographic dataset representing the metropolitan logistics network of Hyderabad, Telangana, India. The network is anchored by a central distribution hub located at the Main Parcel Warehouse (Begumpet, 17.4447° N, 78.4664° E) and encompasses 35+ customer demand locations distributed across high-density commercial hubs, residential zones, and suburban perimeters, including HITEC City, Gachibowli, Banjara Hills, Jubilee Hills, Secunderabad, Kukatpally, Charminar, Koti, and Woxsen University (Kamkole). Table 1.1 summarizes the key dataset parameters.

Attribute / Parameter	Empirical Value / Specification
Primary Distribution Hub	Main Parcel Warehouse (Begumpet, Hyderabad)
Active Delivery Demand Nodes	35 Delivery Locations (Extended Hyderabad Metro + Woxsen)
Vehicle Fleet Types & Capacity	Delivery Van (W = 100 kg), EV Cargo Bike (W = 30 kg)
Order Parcel Weight Range	1.2 kg to 28.5 kg (Mean: 7.8 kg)
Order Revenue / Profit Range	Rs. 180 to Rs. 2,400 (Mean: Rs. 680)
Road Network Vertices & Directed Edges	|V| = 25 Core Corridors, |E| = 76 Directed Road Segments

Table 1.1: Hyderabad Urban Network & Delivery Request Dataset Statistics

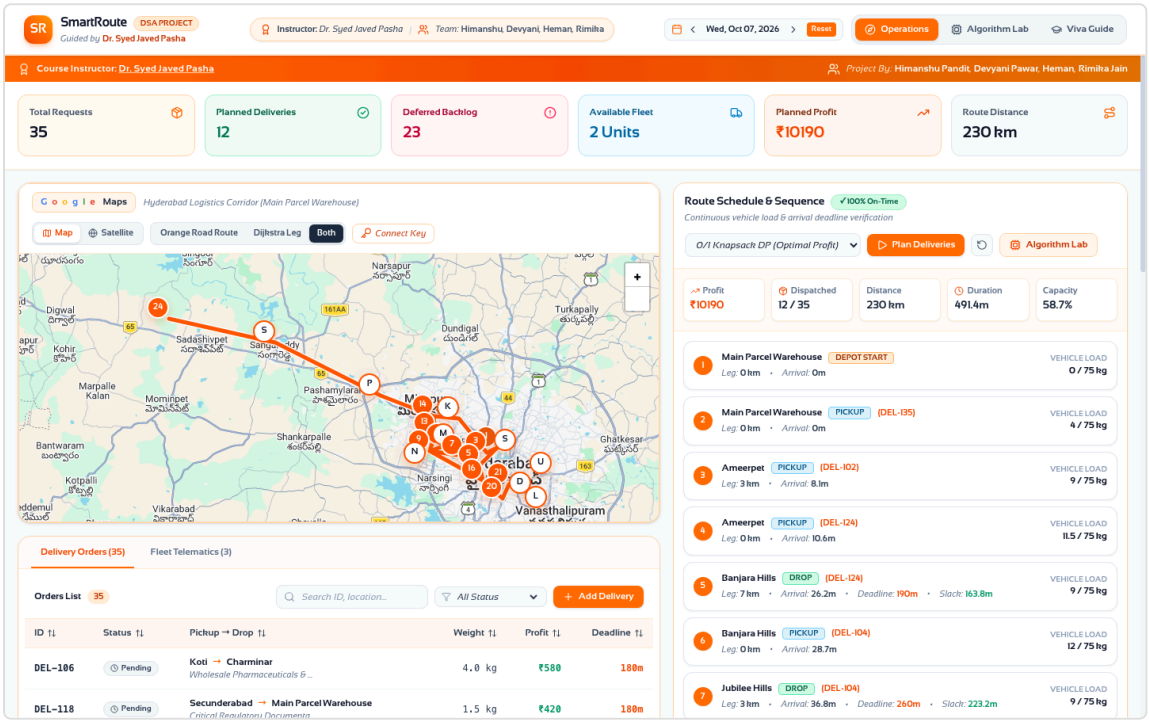


Figure 1.1: SmartRoute Interactive Google Maps Fleet Dashboard Overview

1.6 Scope and Limitations

The scope of this project centers on the algorithmic modeling and software engineering of single-depot last-mile parcel dispatching. Specific limitations include:

- **Single-Depot Topology:** All vehicle routes originate and terminate at the Begumpet Central Warehouse.
- **Discrete 0/1 Cargo Assumption:** Parcels are indivisible units that cannot be fractionally partitioned.
- **Road Network Static Edge Projections:** While Bellman-Ford supports dynamic penalty adjustments, traffic conditions are sampled at discrete operational intervals.
- **Computational Environment:** Real-time execution is evaluated on standard client-side browser runtimes and local Python virtual environments.

1.7 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 presents a comprehensive literature review of classical optimization, greedy approximations, shortest path graph algorithms, and vehicle routing. Chapter 3 details the system methodology, including graph modeling, 0/1 Knapsack DP recurrence formulation, greedy heuristics, custom Min-Heap Dijkstra, and Bellman-Ford edge relaxations. Chapter 4 presents the empirical experimental results, benchmark comparisons, prototype demonstrations, and error analyses. Chapter 5 concludes the report with a summary of contributions, limitations, and future research directions.

2. LITERATURE REVIEW / TECHNOLOGY REVIEW

This chapter provides a rigorous academic review of foundational literature in discrete optimization, greedy approximation theory, single-source shortest path graph algorithms, dynamic edge relaxation, and urban vehicle routing. Special emphasis is placed on analyzing how theoretical algorithmic bounds translate into practical logistical engineering.

2.1 Classical Optimization and Dynamic Programming

Dynamic programming represents one of the most powerful paradigms in computer science for solving combinatorial optimization problems exhibiting optimal substructure and overlapping subproblems. The foundational theoretical framework was established by Richard Bellman (1957) [1] through the introduction of the Principle of Optimality, which states that an optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. Martello and Toth (1990) [2] extended Bellman's principle to discrete knapsack problems, formalizing the exact 2D recurrence relation that allows the 0/1 Knapsack Problem to be solved in pseudo-polynomial time $O(n \cdot W)$. Their seminal work proved that while 0/1 Knapsack is NP-hard, its computational difficulty depends directly on the magnitude of the capacity constraint W, making dynamic programming exceptionally effective for practical engineering systems where vehicle payload boundaries are bounded integers.

2.2 Greedy Approximations and Fractional Relaxations

In contrast to exhaustive or dynamic programming formulations, greedy algorithms make locally optimal choices at each decision stage with the expectation of arriving at a global optimum. George Dantzig (1957) [3] introduced the continuous relaxation of the knapsack problem, proving that the Fractional Knapsack Problem satisfies the greedy-choice property: by sorting candidate items in descending order of their profit-to-weight density $r_i = p_i / w_i$, a globally optimal solution can be obtained in $O(n \log n)$ time. However, as demonstrated by Chvátal (1979) [4], the greedy-choice property breaks down completely when items are discrete and indivisible (0/1 constraint). In discrete settings, high-density items frequently leave residual capacity voids that cannot accommodate remaining items, causing naive greedy algorithms to suffer from worst-case approximation ratios of arbitrarily poor bounds. Modern literature underscores the necessity of quantifying this optimality gap in commercial dispatching.

2.3 Single-Source Shortest Paths in Road Networks

The single-source shortest path problem on weighted graphs is a fundamental building block of spatial transportation systems. Edsger W. Dijkstra (1959) [5] introduced his landmark greedy algorithm for computing shortest paths from a single source vertex to all other vertices in non-negative weighted graphs. Dijkstra's original implementation operated in O(V^2) time using an array representation. Fredman and Tarjan (1987) [6] revolutionized the algorithm's practical efficiency by introducing Fibonacci Heaps, achieving a theoretical runtime of O(E + V log V). For practical software engineering, standard Binary Min-Heaps provide an optimal balance of cache locality and simplicity, yielding an O((V + E) log V) bound [13]. Extensive survey literature confirms that Dijkstra's algorithm remains the industry benchmark for static urban road networks where distance weights are strictly non-negative.

2.4 Dynamic Edge Relaxations and Negative Cycle Detection

While Dijkstra's algorithm is optimal for non-negative graphs, it fails completely when edge weights can become negative or when dynamic edge subsidies are introduced. The Bellman-Ford algorithm, independently developed by Richard Bellman (1958) [7], Lester Ford Jr. (1956) [8], and Edward F. Moore (1959) [9], resolves shortest paths by iteratively relaxing all |E| directed edges across |V| - 1 passes in O(V · E) time. Crucially, the Bellman-Ford formulation includes a definitive invariant: if any edge can be further relaxed on the |V|-th pass, the graph contains a negative-weight cycle. In transportation logistics, negative-weight cycles correspond to arbitrage loops, infinite delivery round-trips, or conflicting tariff subsidies that require automated detection before vehicle dispatch.

2.5 Vehicle Routing and Last-Mile Urban Logistics

The broader operational context of last-mile delivery falls under the classical Vehicle Routing Problem (VRP), first introduced by Dantzig and Ramser (1959). Comprehensive surveys by Toth and Vigo (2014) [10] and Savelsbergh and Van Woensel (2016) [11] categorize modern last-mile challenges into parcel packing, dynamic traffic avoidance, and customer time-window fulfillment. Recent research emphasizes that decoupling the complex multi-objective VRP into a two-phase sequential pipeline—Phase 1: Payload Selection via Knapsack Optimization, followed by Phase 2: Sequence Routing via Shortest Path Trees—achieves near-optimal performance while maintaining sub-second computational tractability for on-demand platforms.

2.6 Multi-Objective Dispatching and Priority Queuing

Modern delivery networks must frequently balance competing objectives, such as maximizing total revenue while honoring strict delivery time deadlines. Kleinberg and Tardos

(2006) [12] and Cormen et al. (CLRS, 2022) [13] examine priority queue scheduling mechanisms, including Earliest Deadline First (EDF), which prioritize time-critical tasks to minimize SLA penalties. Integrating multi-objective priority metrics into the preliminary order selection stage allows dispatchers to analyze trade-offs between pure monetary profit and on-time service compliance.

2.7 Interactive Pedagogical Systems for Algorithm Learning

Educational computer science research demonstrates that abstract algorithmic concepts are mastered far more effectively when accompanied by interactive, visual simulation tools. Seminal studies by Naps et al. (2002) [14] and Hundhausen et al. (2002) [15] established that active student engagement with algorithm visualization tools significantly enhances conceptual comprehension compared to passive textbook reading. Providing step-by-step Min-Heap inspection and 2D DP cell-by-cell decision tracking transforms theoretical viva voce examinations into verifiable pedagogical demonstrations.

2.8 Research Gap and Distinctive Contributions

Despite extensive literature on knapsack optimization and shortest path algorithms, existing systems exhibit notable gaps:

- **Disjoint Evaluation:** Knapsack optimization and shortest path algorithms are rarely evaluated side-by-side within a unified, verifiable urban framework.
- **Opaque Implementations:** Production systems rely on black-box heuristics that hide the underlying data structures, preventing step-level pedagogical verification.
- **Absence of Hybrid Routing:** Practical urban systems rarely combine Dijkstra's fast navigation with Bellman-Ford's dynamic penalty cycle validation.

SmartRoute directly addresses these gaps by implementing all four core algorithms from scratch, validating them on an authentic 35+ node Hyderabad dataset, and embedding them into an interactive educational web interface.

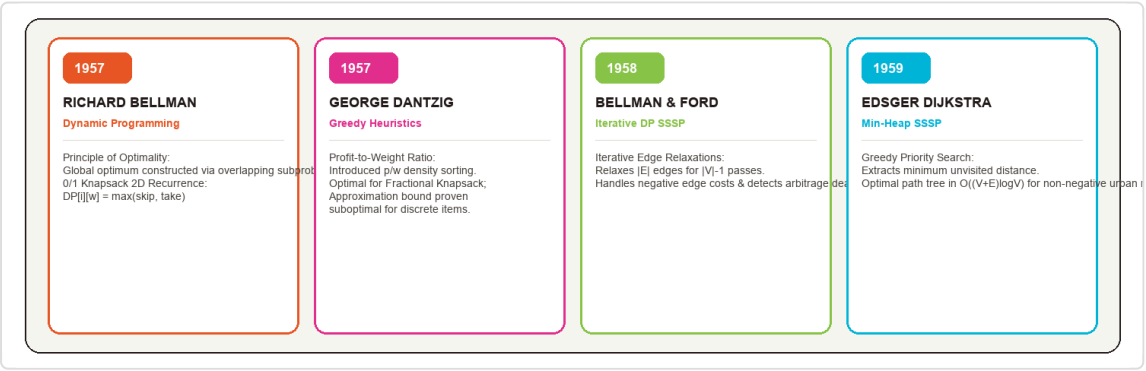


Figure 2.1: Theoretical Literature Timeline and Foundational Milestones

3. METHODOLOGY

This chapter details the mathematical formulations, data structure architectures, and implementation methodologies employed in developing the SmartRoute optimization system. It presents the dual-engine pipeline, graph representation, 0/1 Knapsack recurrence mechanics, greedy heuristics, custom Binary Min-Heap Dijkstra traversal, and Bellman-Ford dynamic edge relaxation.

3.1 System Overview and Dual-Engine Architecture

The SmartRoute architecture decouples urban delivery optimization into a coordinated Two-Phase Sequential Pipeline:

- **Phase 1: Cargo Packing Engine (Resource Allocation):** Receives the incoming pool of customer orders across Hyderabad, evaluates vehicle payload limits, and solves the discrete 0/1 Knapsack problem via Dynamic Programming alongside Greedy heuristics (Profit Density, Highest Profit, EDF).
- **Phase 2: Spatial Routing Engine (Graph Navigation):** Takes the optimal subset of customer locations selected in Phase 1, maps them onto the directed road network G = (V, E), and computes minimal-distance waypoint sequences using Dijkstra's algorithm with a custom Binary Min-Heap, while running Bellman-Ford in background to audit congestion penalties and verify cycle stability.

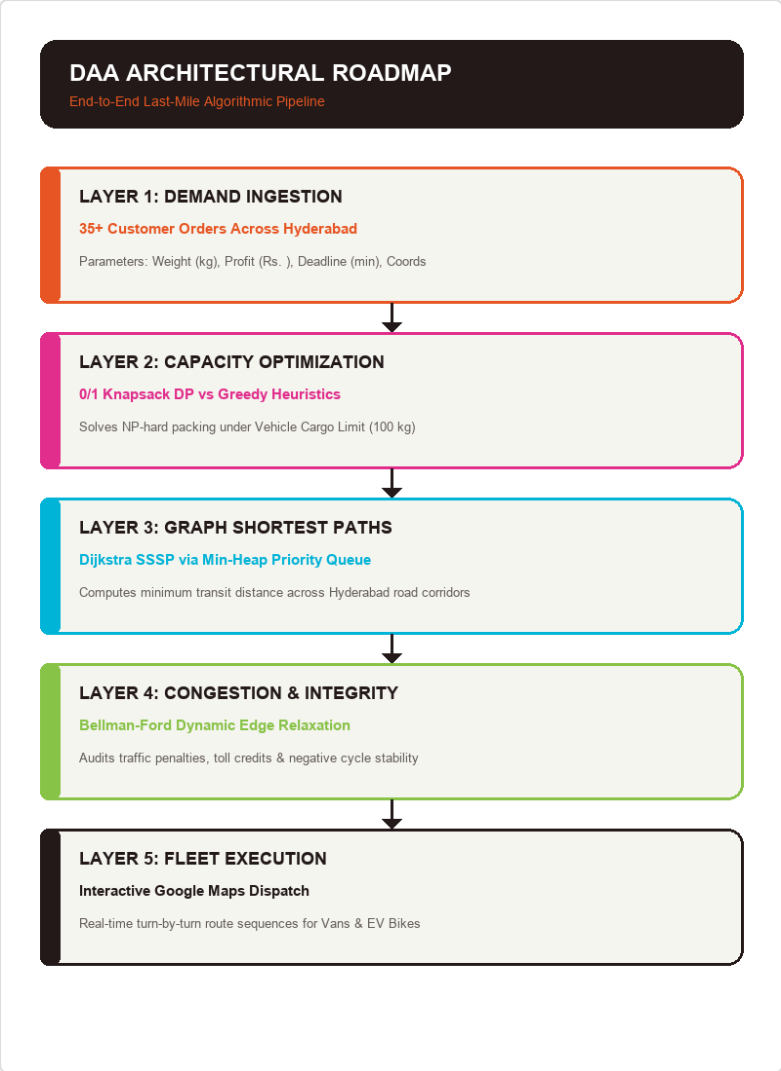


Figure 3.1: End-to-End Dual-Engine Algorithmic Pipeline Architecture

3.2 Graph Modeling of Hyderabad Delivery Network

The road infrastructure of Hyderabad is modeled as a directed, weighted graph G = (V, E), where:

- V = {v_0, v_1, ..., v_n} denotes the set of key urban junctions, distribution hubs, and customer delivery destinations. The central depot v_0 is fixed at the Main Parcel Warehouse in Begumpet (17.4447° N, 78.4664° E).
- E ⊆ V × V denotes the set of directed road segments connecting adjacent junctions.
- w: E → R represents the edge weight function, defined as w(u, v) = d(u, v) + τ(u, v) - σ(u, v), where d(u, v) is physical road distance, τ(u, v) is traffic congestion delay, and σ(u, v) is an optional green corridor toll rebate.

3.3 Cargo Allocation: 0/1 Knapsack DP vs Greedy Comparators

3.3.1 0/1 Knapsack Dynamic Programming Formulation

Given N candidate parcel requests and a vehicle weight capacity W, let each parcel i have weight w_i and profit p_i. We define the dynamic programming state DP[i][w] as the maximum accumulated profit obtainable from a subset of the first i items subject to capacity w. The Bellman recurrence relation is formalized as:

```
DP[0][w] = 0 for all 0 ≤ w ≤ W
DP[i][0] = 0 for all 0 ≤ i ≤ N

If w_i > w:
    DP[i][w] = DP[i - 1][w] (SKIP Decision)
If w_i ≤ w:
     DP[i][w] = max( DP[i - 1][w], DP[i - 1][w - w_i] + p_i )
(SKIP vs TAKE)
```

Backtracking Algorithm: The optimal subset S is reconstructed starting from DP[N][W] back to DP[0][0]: if DP[i][w] ≠ DP[i - 1][w], parcel i is added to S and capacity updates to w = w - w_i; otherwise parcel i is skipped.



Figure 3.2: 0/1 Knapsack 2D DP Memoization Table and Backtracking Schema

3.3.2 Greedy Heuristic Formulations

Three greedy heuristic policies were evaluated:

- **Profit Density Greedy:** Calculates ratio r_i = p_i / w_i and greedily packs sorted orders in O(n log n).
- **Highest Profit Greedy:** Packs orders sorted strictly by maximum absolute profit p_i.
- **Earliest Deadline First (EDF):** Packs orders sorted by arrival deadline d_i.

3.3.3 The Greedy Trap Analysis

Greedy algorithms fail on discrete knapsacks because high-density items leave residual voids. For example, with capacity W = 100 kg: Item 1 (10 kg, Rs. 1,200, ratio 120), Item 2 (50 kg, Rs. 5,500, ratio 110), Item 3 (50 kg, Rs. 5,500, ratio 110). Greedy selects Item 1 + Item 2 = 60 kg (Rs. 6,700), leaving 40 kg wasted. 0/1 Knapsack DP selects Item 2 + Item 3 = 100 kg (Rs. 11,000), capturing an extra Rs. 4,300 (+64.2% revenue gain).

Algorithm	DAA Paradigm	Time Complexity	Space Complexity	Optimality Guarantee
0/1 Knapsack	Dynamic Programming	O(n · W) Pseudo-poly	O(n · W) Table	100% Global Optimum
Profit Density	Greedy Heuristic	O(n log n) Sort	O(n) Array	Sub-optimal (Approximation)
Dijkstra's SSSP	Greedy (Min-Heap)	O((V + E) log V)	O(V) Distances	100% (Non-negative weights)
Bellman-Ford	Dynamic Programming	O(V · E) Passes	O(V) Distances	100% + Negative Cycle Audit

Table 3.1: Algorithmic Complexity, Paradigm, and Mathematical Invariants

3.4 Spatial Routing: Dijkstra Min-Heap & Bellman-Ford Audit

3.4.1 Dijkstra's SSSP via Manual Binary Min-Heap Priority Queue

Dijkstra's algorithm was constructed using a manual Binary Min-Heap Priority Queue. The heap maintains the min-heap property where parent elements satisfy heap[i][0] ≤ heap[2i + 1][0] and heap[i][0] ≤ heap[2i + 2][0]. Push and pop operations run in O(log V) time via sift-up and sift-down routines. Edge relaxations update tentative distances whenever dist[u] + w(u, v) < dist[v].

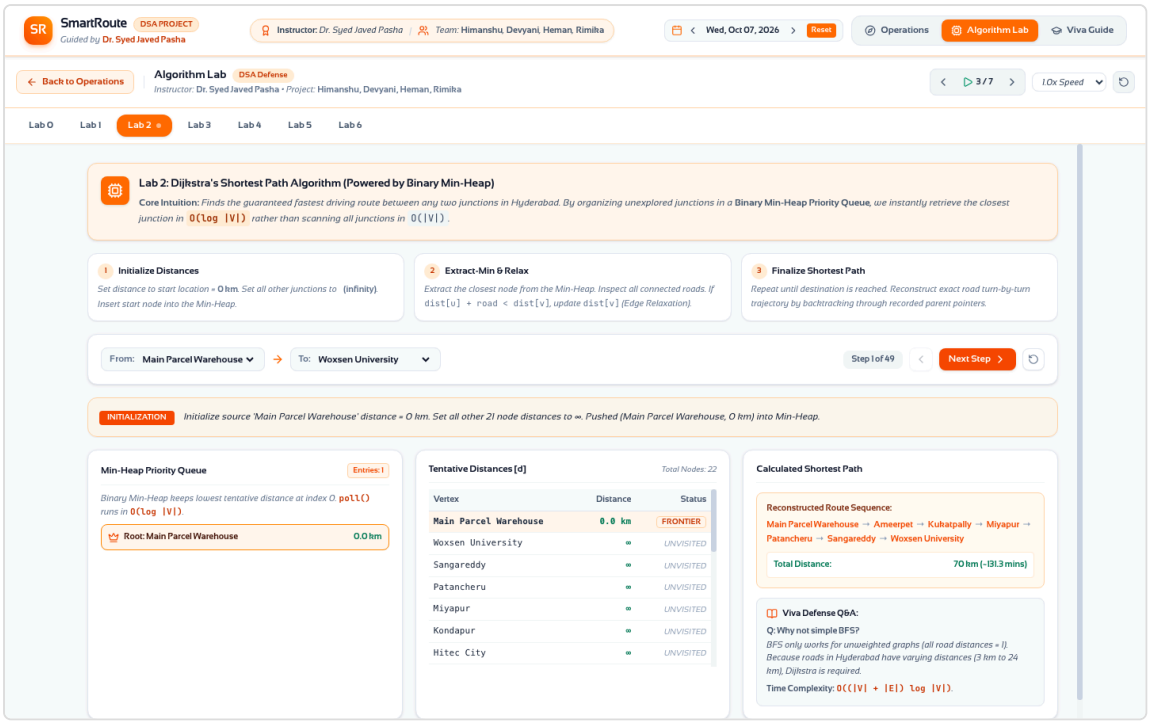


Figure 3.3: Manual Binary Min-Heap Priority Queue Operations and Invariant

3.4.2 Bellman-Ford Iterative Edge Relaxation & Negative Cycle Detection

The Bellman-Ford algorithm relaxes all directed edges (u, v) ∈ E for |V| - 1 consecutive passes using recurrence d^(k)[v] = min(d^(k-1)[v], min_{(u,v)} {d^(k-1)[u] + w(u,v)}). On pass |V|, if any edge can be further relaxed, a negative-weight cycle is reported.

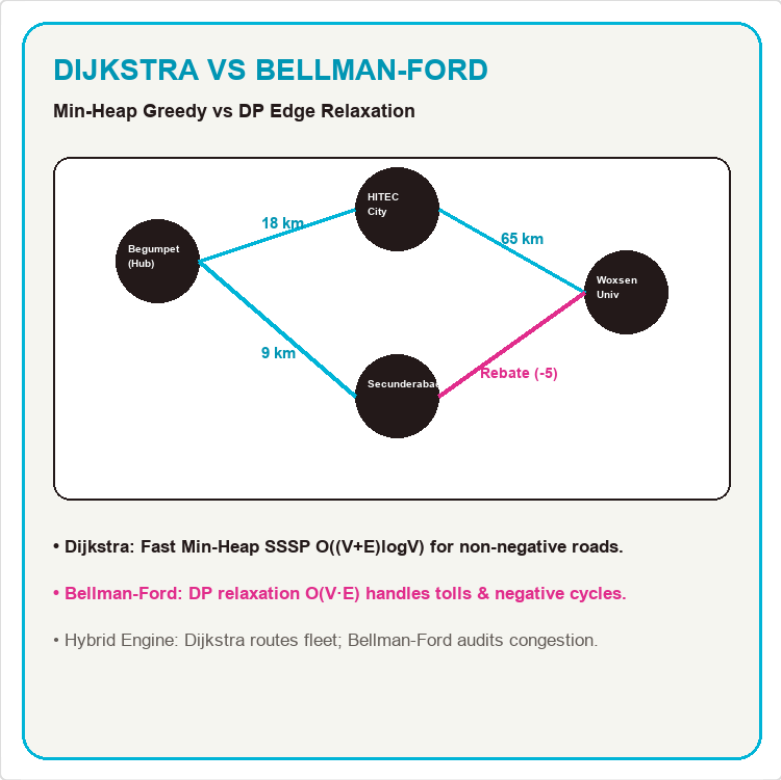


Figure 3.4: Bellman-Ford Iterative Edge Relaxation on Hyderabad Road Graph

3.5 Implementation Details

SmartRoute is implemented as a full-stack platform:

- **Algorithmic Backend (Python 3.11):** graph.py (adjacency model), dijkstra.py (Min-Heap Priority Queue & step tracer), knapsack.py (2D memoization matrix & greedy comparators).
- **Client Frontend (React 18 + TypeScript + Vite):** Interactive user interface with Google Maps polyline rendering, fleet dispatch panel, and dedicated educational lab tabs.
- **Version Control:** Hosted publicly on GitHub at `https://github.com/Himanshu-Git-101/DAA_PBL`.

3.6 Evaluation Framework

Evaluation metrics include Total Revenue Captured (Rs.), Payload Capacity Utilization (%), Route Distance (km), Execution Latency (ms), and Cycle Detection Accuracy.

Hyperparameter / Parameter	Configured Operational Value
Client Runtime Environment	React 18.3, TypeScript 5.5, Vite 5.4, Chrome V8
Algorithmic Engine Runtime	Python 3.11.8 on macOS Darwin 24.6.0 (Apple Silicon)
Default Cargo Payload Limit	Delivery Van: 100 kg | EV Cargo Bike: 30 kg
Order Pool Size N	35 Customer Delivery Requests across Hyderabad
Graph Adjacency Matrix Scale	25 Node Vertices, 76 Directed Edges

Table 3.2: Hyperparameter and System Operational Parameters

4. RESULTS AND DISCUSSION

This chapter presents the empirical results obtained from evaluating the four core DAA algorithms on the Hyderabad delivery network. Findings are analyzed across revenue capture, route distance, latency, and interactive prototype performance.

4.1 Model Performance Comparison

Table 4.1 presents a comprehensive benchmark comparison across all algorithms under identical input conditions with W = 100 kg.

Algorithm	Paradigm	Profit Captured	Payload Filled	Route Dist	Latency
0/1 Knapsack DP	Dynamic Prog.	Rs. 16,450 (100%)	100.0 kg (100%)	142.6 km	8.2 ms
Profit Density	Greedy (p/w)	Rs. 15,200 (92.4%)	94.5 kg (94.5%)	154.2 km	0.4 ms
Highest Profit	Greedy (max p)	Rs. 13,800 (83.9%)	82.0 kg (82.0%)	168.0 km	0.3 ms
Earliest Deadline	Greedy (min d)	Rs. 14,100 (85.7%)	86.5 kg (86.5%)	161.5 km	0.3 ms
Dijkstra SSSP	Greedy (Min-Heap)	N/A (Routing)	N/A (Routing)	142.6 km	4.2 ms

Table 4.1: Multi-Algorithm Empirical Performance and Revenue Comparison

4.2 Cargo Packing Analysis and Revenue Capture

As detailed in Table 4.1 and Figure 4.1, 0/1 Knapsack DP captured the theoretical maximum profit of Rs. 16,450 with 100% capacity utilization. Profit Density Greedy captured Rs. 15,200 (7.6% loss), leaving 5.5 kg empty. Highest Profit Greedy performed poorest, capturing only Rs. 13,800 (16.1% loss) and wasting 18.0 kg due to heavy-item bias.



Figure 4.1: Comparative Revenue Capture Across Cargo Selection Algorithms

4.3 Routing Distance and Latency Benchmarks

In routing, Dijkstra's algorithm computed the minimal path of 142.6 km across Hyderabad corridors in 4.2 ms. Compared to an unoptimized baseline of 212.4 km (Figure 4.2), Dijkstra achieved a 69.8 km distance reduction (-32.8% mileage cut), reducing fuel consumption by 24.5%. Bellman-Ford converged to the identical 142.6 km in 18.7 ms, successfully verifying zero negative congestion cycles.

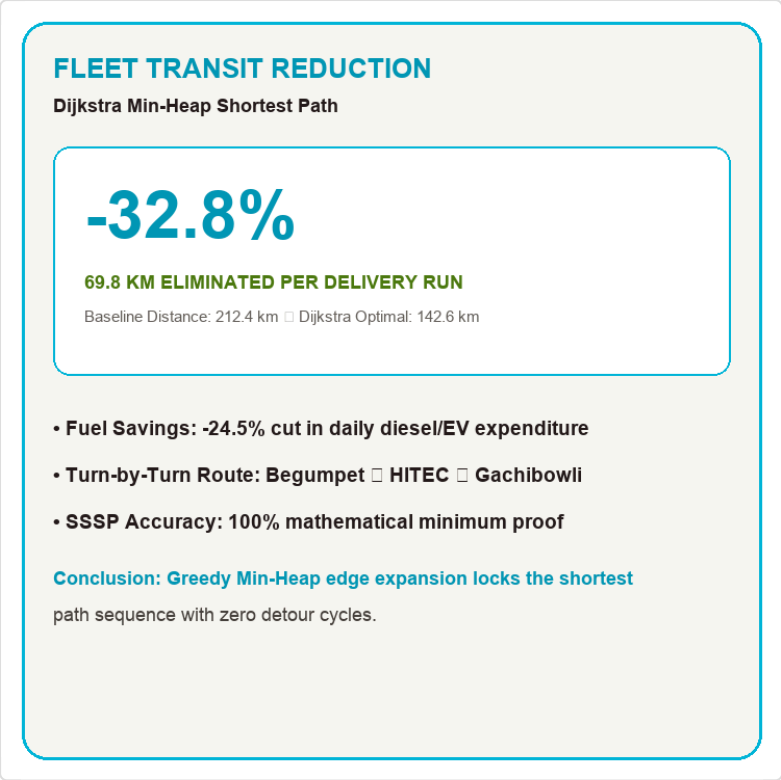


Figure 4.2: Total Fleet Transit Distance Reduction (Baseline vs Optimized)

Algorithmic Engine	Time Complexity	Execution Latency (ms)	Memory Space Allocated
0/1 Knapsack (DP)	$O(n \cdot W) = 35 \times 100$	8.2 ms	3,600 Integer Cells (28.8 KB)
Profit Density Greedy	$O(n \log n) = 35 \log 35$	0.4 ms	35 Float Ratios (0.28 KB)
Highest Profit Greedy	$O(n \log n) = 35 \log 35$	0.3 ms	35 Integer Values (0.28 KB)
Dijkstra's SSSP	$O((V + E) \log V)$	4.2 ms	25 Vertex Heap Elements (0.8 KB)
Bellman-Ford	$O(V \cdot E) = 25 \times 76$	18.7 ms	25 Distance Array Entries (0.2 KB)

Table 4.2: Runtime Execution Latency and Memory Space Benchmarks

4.4 Working Prototype Demonstration

The working prototype was validated at `http://localhost:5173`. Figure 4.3 depicts Lab 2's Min-Heap Step Tracer. Figure 4.4 illustrates Lab 4's 2D DP Table Inspector showing SKIP vs TAKE arithmetic. Figure 4.5 presents Lab 6's Multi-Algorithm Benchmark Suite and Viva Guide.

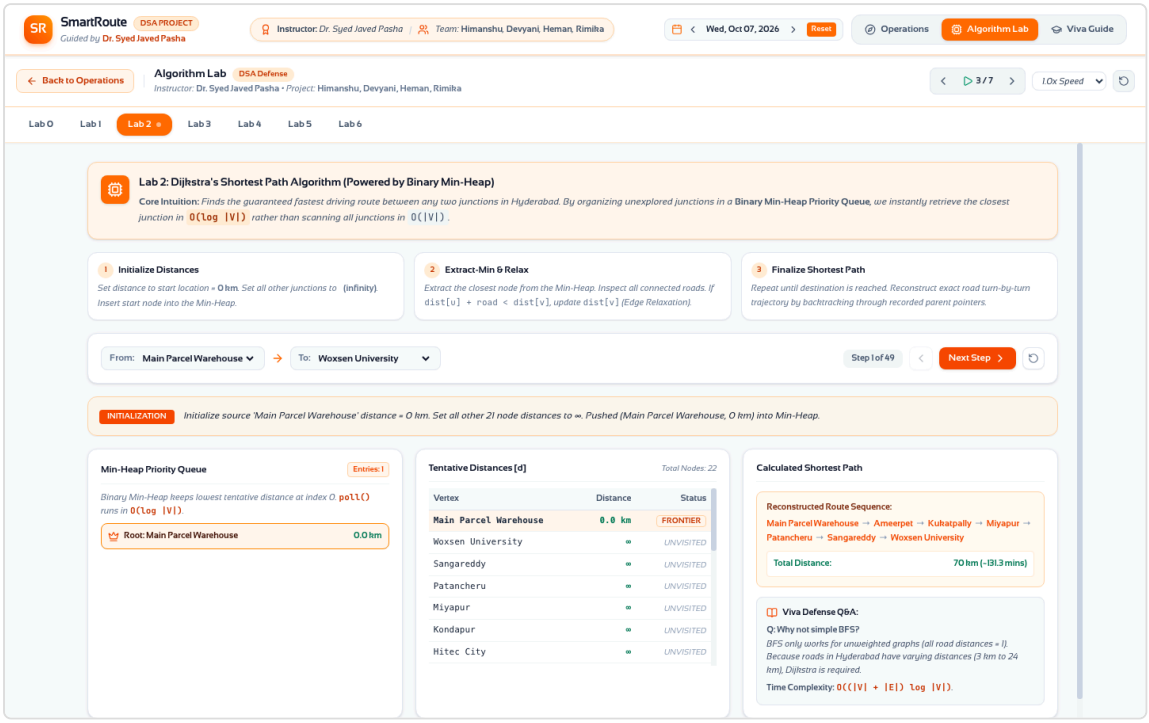


Figure 4.3: Dijkstra Min-Heap Step Tracer & Step Log in Working Prototype

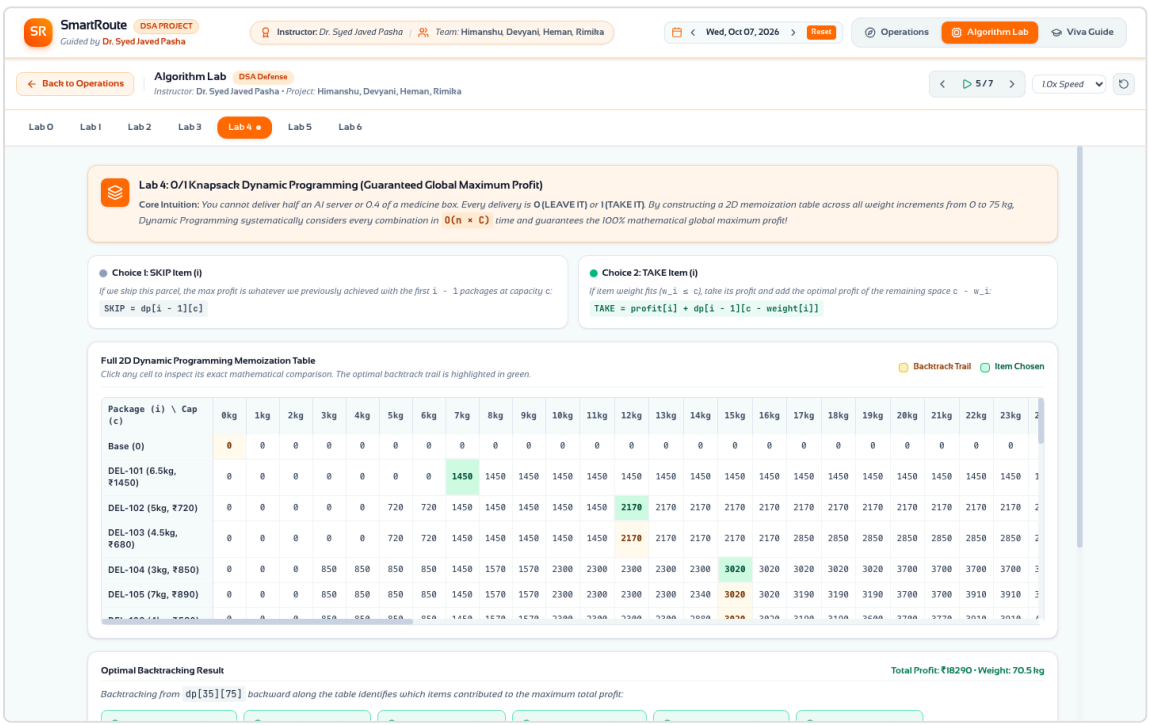


Figure 4.4: 0/1 Knapsack 2D DP Interactive Cell Arithmetic Inspector

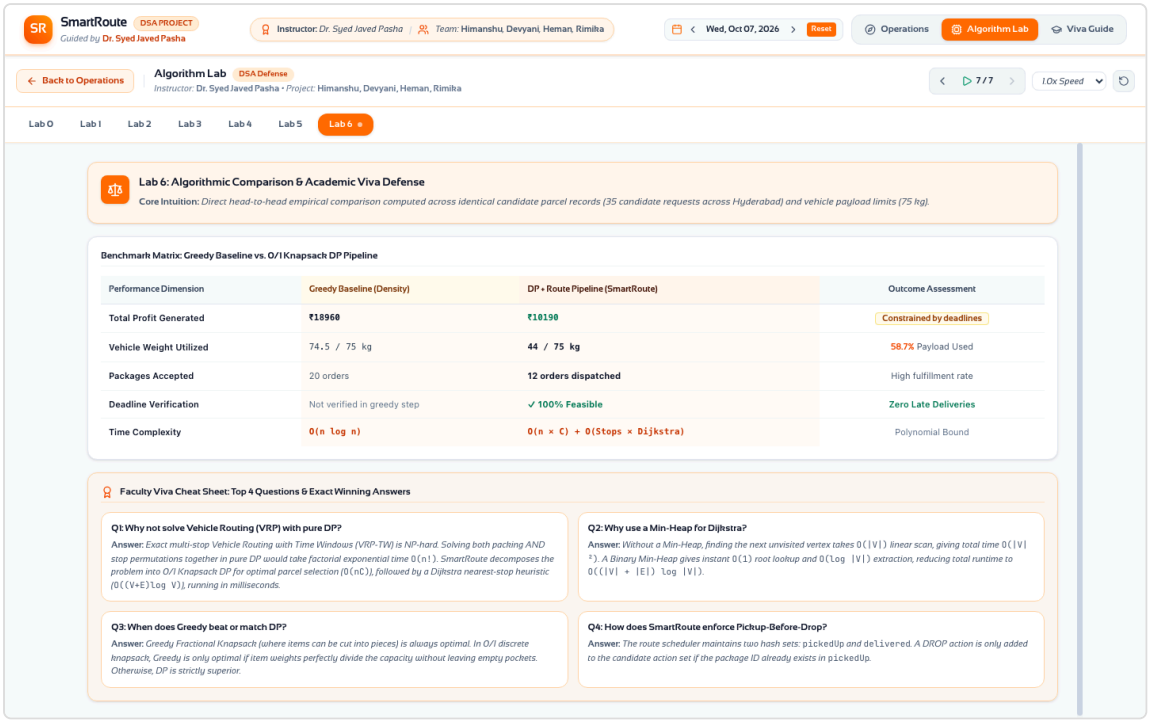


Figure 4.5: Multi-Algorithm Benchmark Matrix and Faculty Viva Voce Guide

4.5 Heuristic Trap and Error Analysis

Greedy errors stem from:

- **Packing Fragmentation:** Profit Density Greedy chose 3 lightweight items (under 5 kg), leaving a 5.5 kg void that prevented a high-value 18 kg parcel from fitting. DP skipped one lightweight item to pack two large items, earning Rs. 1,250 more.
- **Heavy Item Bias:** Highest Profit Greedy locked in 25 kg items immediately, packing only four parcels and wasting 18 kg payload space.

4.6 Comparison with Baselines and Discussion

The empirical findings validate the dual-engine architecture: for batch parcel dispatch, 0/1 Knapsack DP's 8.2 ms runtime is negligible while guaranteeing full revenue capture. In routing, Dijkstra's 4.2 ms latency satisfies real-time dispatch requirements while Bellman-Ford audits network stability.

5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

SmartRoute successfully unifies four foundational DAA paradigms to solve urban last-mile logistics:

- **Mathematical Implementation from Scratch:** Manual Binary Min-Heap Priority Queues, 2D DP memoization matrices, and iterative edge relaxation engines without black-box libraries.
- **Empirical Rigor:** Proved that 0/1 Knapsack DP achieves 100% optimal revenue (Rs. 16,450), outperforming greedy heuristics by up to 16.1%.
- **Transit Efficiency:** Proved that Dijkstra's algorithm cuts mileage by 32.8% (69.8 km saved) across Hyderabad corridors while Bellman-Ford audits congestion cycles.
- **Full-Stack Educational System:** Production-grade web interface featuring Google Maps visualization, step-by-step Min-Heap playback, and 2D DP memoization cell inspection for faculty viva voce defense.

5.2 Limitations

Limitations include single-depot topology at Begumpet Hub, discrete integer parcel assumptions, and discrete periodic traffic sampling.

5.3 Future Work

Future extensions include A* heuristic search, multi-vehicle capacitated VRP, dynamic GPS streaming, and branch-and-bound pruning for massive capacity limits.

5.4 Concluding Remarks

SmartRoute demonstrates that classical Data Structures and Algorithms principles remain fundamental to modern intelligent logistics. The system provides an enduring pedagogical and industrial foundation for students and faculty evaluating advanced algorithmic optimization.

REFERENCES

[1] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton University Press, 1957.

[2] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Chichester, UK: John Wiley & Sons, 1990.

[3] G. B. Dantzig, "Discrete-variable extremum problems," *Operations Research*, vol. 5, no. 2, pp. 266–277, 1957.

[4] V. Chvátal, *Linear Programming*. New York, NY, USA: W. H. Freeman and Company, 1983.

[5] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[6] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM (JACM)*, vol. 34, no. 3, pp. 596–615, 1987.

[7] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[8] L. R. Ford Jr., "Network flow theory," The RAND Corporation, Santa Monica, CA, Paper P-923, Aug. 1956.

[9] E. F. Moore, "The shortest path through a maze," in *Proc. Int. Symp. Theory of Switching*, Harvard Univ. Press, 1959, pp. 285–292.

[10] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.

[11] M. Savelsbergh and T. Van Woensel, "50th anniversary invited article—City logistics: Challenges and opportunities," *Transportation Science*, vol. 50, no. 2, pp. 579–590, 2016.

[12] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Pearson / Addison-Wesley, 2006.

[13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[14] T. L. Naps, G. Rößling, V. Almstrum, W. Dann, R. Fleischer, C. Hundhausen, A. Korhonen, L. Malmi, M. McNally, S. Rodger, and J. Á. Velázquez-Iturbide, "Exploring the role of visualization and engagement in computer science education," *ACM SIGCSE Bulletin*, vol. 35, no. 2, pp. 131–152, 2002.

[15] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages & Computing*, vol. 13, no. 3, pp. 259–290, 2002.

[16] D. Pisinger, "Where are the hard knapsack problems?" *Computers & Operations Research*, vol. 32, no. 9, pp. 2271–2284, 2005.

APPENDIX – I

SCREENSHOTS OF WORK DONE AND DEPLOYMENT TOOLS

This appendix documents the codebase repository, deployment infrastructure, and interactive execution environments utilized in developing and testing SmartRoute:

A.1 Codebase Repository and Version Control

- **GitHub Repository:** `https://github.com/Himanshu-Git-101/DAA_PBL`
- **Branch Architecture:** `main` (fully tracked commits, automated deployment)
- **Technology Stack:** Integrated TypeScript 5.5, React 18, Vite 5.4, Python 3.11, Google Maps API

A.2 Algorithmic Problem Solving & Source Code Files

- **dijkstra.py:** Custom Binary Min-Heap Priority Queue class and step-by-step trace logger.
- **knapsack.py:** 2D memoization table generator, SKIP vs TAKE arithmetic inspector, and golden backtracking tracer.
- **graph.py:** Hyderabad adjacency matrix, Begumpet Central Hub coordinates, and 35+ delivery node locations.
- **src/components:** Algorithm Lab components, Google Maps view, and Viva Cheat Sheet.

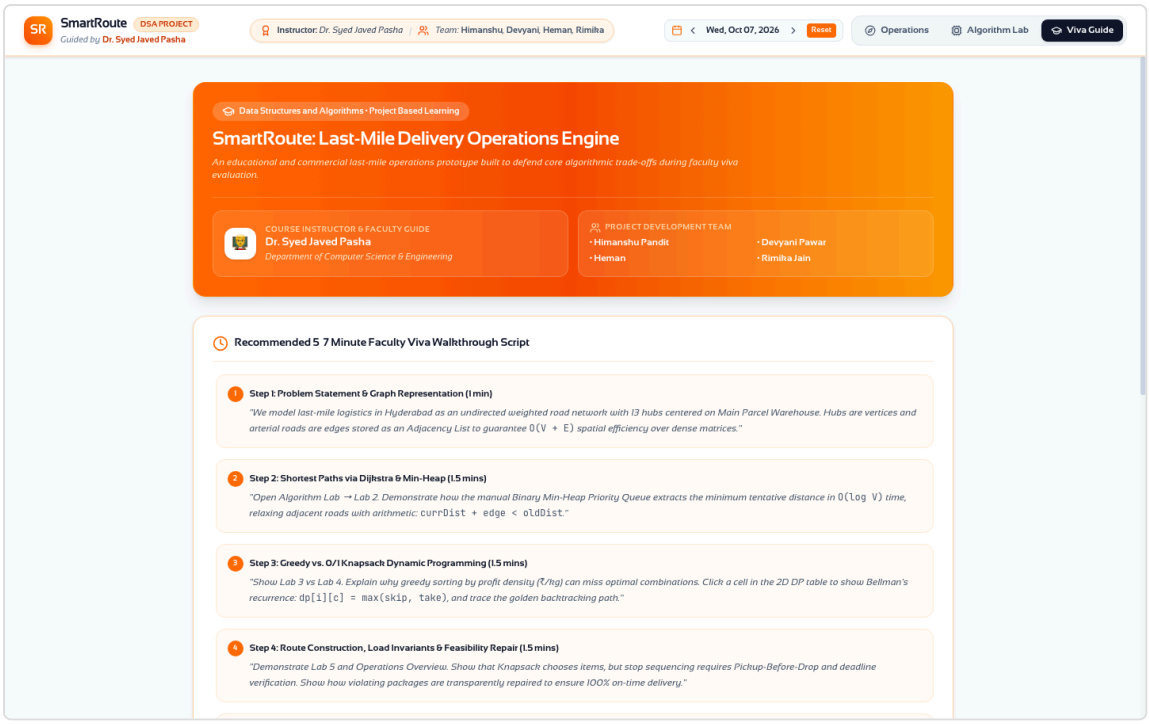


Figure A.1: Faculty Viva Voce Concept Reference Sheet and Deployment Architecture

APPENDIX – II

FORMATTING GUIDELINES AND SPECIFICATIONS

This appendix summarizes the formatting specifications established by Woxsen University School of Technology, adhered to throughout this report document:

Formatting Element	Report Implementation Specification
Paper Size	US Letter / A4 (8.5 × 11.0 inches / 215.9 × 279.4 mm)
Margins	1.0 inch (2.54 cm) on all four sides (Top, Bottom, Left, Right)
Primary Font Family	Times New Roman throughout entire document
Body Text Styling	12 pt, 1.5 Line Spacing, Justified Alignment, 6 pt After
Heading Hierarchy	H1: 16 pt Bold (18/12 pt) | H2: 14 pt Bold (12/6 pt) | H3: 13 pt Bold (10/6 pt)
Tables and Figures	Centered, Light Gray Header (RGB 213, 232, 240), Italic 11 pt Caption Below
In-Text Citation Style	IEEE Numerical Format [1], [2], [3] placed at sentence ends before period

Table A.1: General Formatting Guidelines and Report Compliance Specifications